

01211373 Machine Learning and Programming for Industry

DEEP LEARNING

Attention & Transformers

Lecture 11 — Finding the Needle in a Long Sequence

Last Week → This Week

WEEK 10 RECAP

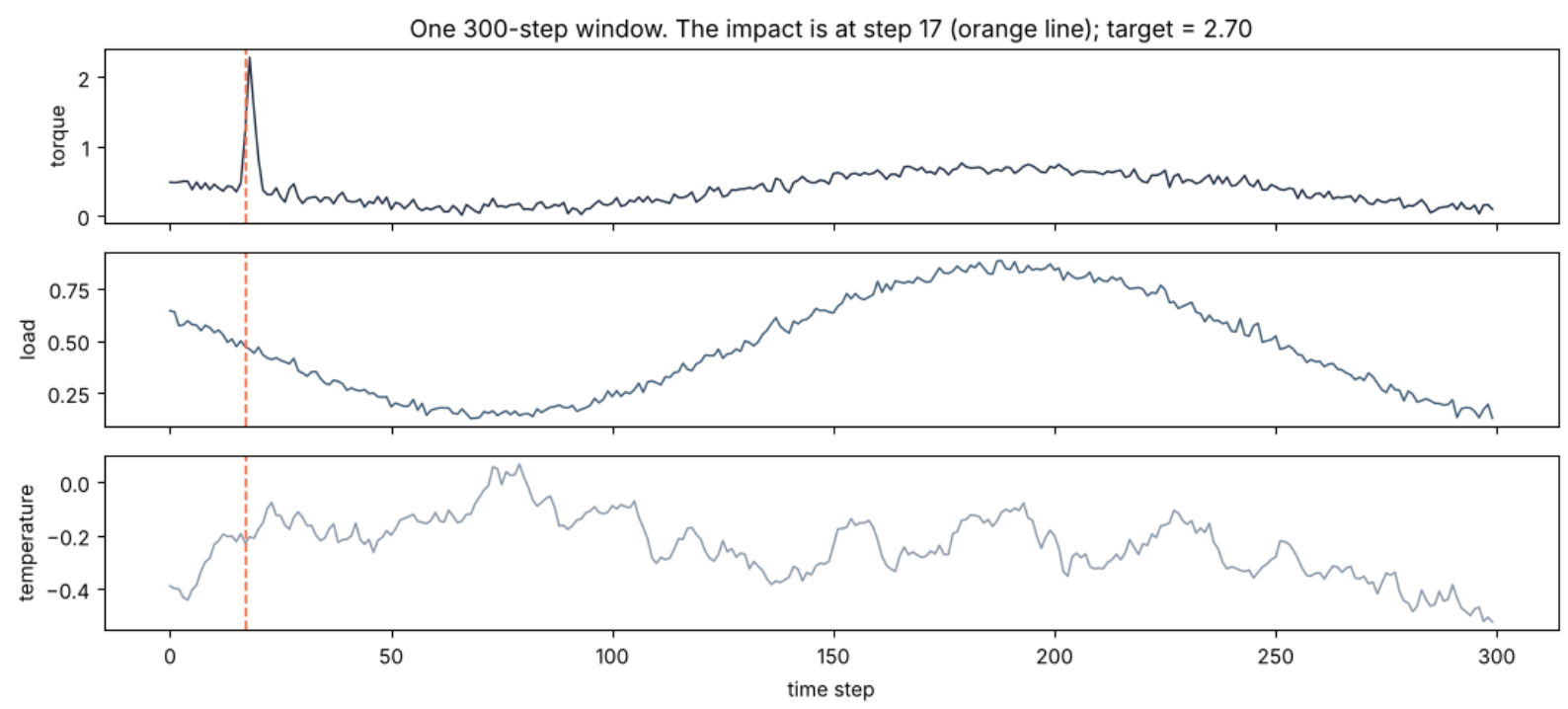
- A hidden state carries the past forward
- LSTM gates keep gradients alive
- Shuffling the input proved the model used order
- But everything must squeeze through one fixed-size state

TODAY

- Remove the bottleneck: look anywhere, directly
- Queries, keys and values
- Scaled dot-product attention, the one equation
- Multi-head attention and position encoding
- Where transformers sit in robotics perception

Today's Problem: A Brief Impact in a Long Log

A robot joint takes an impact. How much damage it causes depends on the impact magnitude AND the load at that same instant.



The signal

4 samples of torque transient

The haystack

300 samples of telemetry

The catch

must read a 2nd channel at the same instant

The model must FIND the event, then read two channels there.

The Bottleneck in a Recurrent Network

Whatever the LSTM notices at step p must survive, inside one fixed-size vector, all the way to the end of the sequence.



everything the model remembers travels left-to-right through this one narrow channel

Short sequence: no problem. The state only has to hold on for a few steps.

Long sequence: the state is overwritten hundreds of times before the answer is needed — and the gradient has that far to travel back.

Attention removes the channel entirely. Instead of passing information along, it lets the output reach back and read any position directly.

The Idea: A Soft Lookup Table



Attention is a dictionary lookup where the match is graded rather than exact. Every position offers a key and a value; the query says what we are looking for.



Query

What am I looking for?

In our model, one learned vector — effectively the question “where is the impact?”



Keys

What does each position advertise?

One key per time step, used only for matching against the query.



Values

What does each position actually offer?

The content that gets returned, weighted by how well its key matched.

The whole mechanism in one sentence: score the query against every key, turn the scores into weights that sum to one, and return that weighted average of the values.

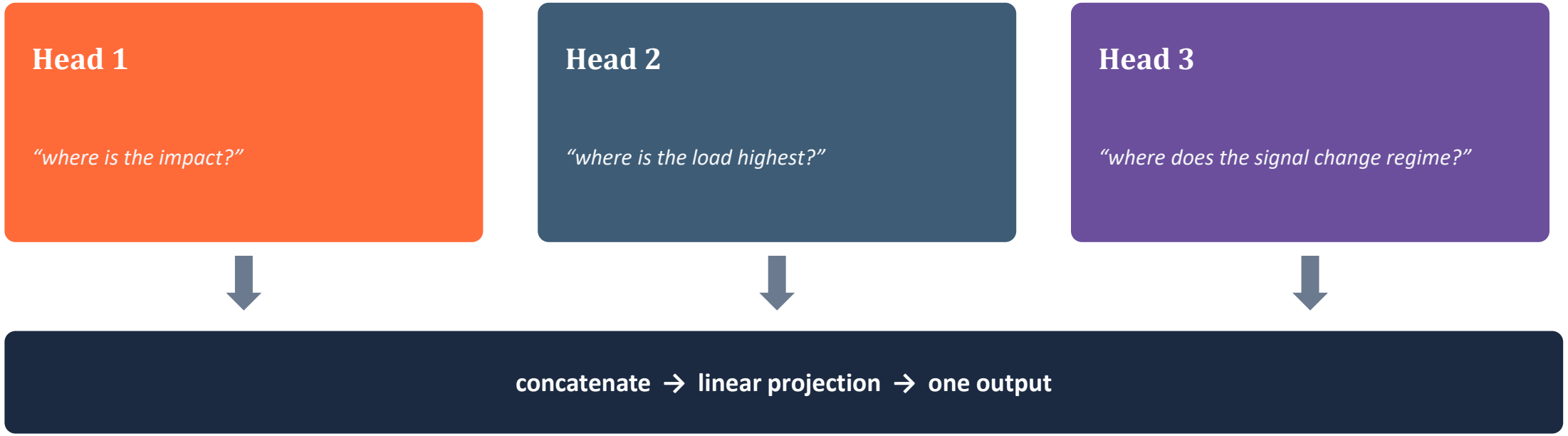
Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q K^T / \sqrt{d_k} \right) V$$

1	`Q × K^T`	Dot every query against every key. A large dot product means the key points the same way as the query — a good match.
2	$\div \sqrt{d_k}$	With d_k large, dot products grow large, pushing softmax to nearly one-hot and its gradient to nearly zero. Dividing rescales them back.
3	`softmax`	Turn raw scores into weights between 0 and 1 that sum to one — a distribution over positions.
4	`× V`	Weighted average of the values. Positions that matched well dominate the result.

Multi-Head Attention

One query asks one question. Real transformers run several attention operations in parallel, each with its own learned projections, then concatenate the results.



The head labels above are illustrative — nobody assigns them. Each head learns whatever projection reduces the loss.

Attention Is Order-Blind by Default



Shuffle the time steps and the set of keys and values is unchanged — so the output is unchanged. This is the exact opposite of Week 10's LSTM.

Sometimes that is fine

Our task asks only how big the impact was and what the load was at that moment. Neither depends on WHERE in the window it happened, so our model needs no positional information at all.

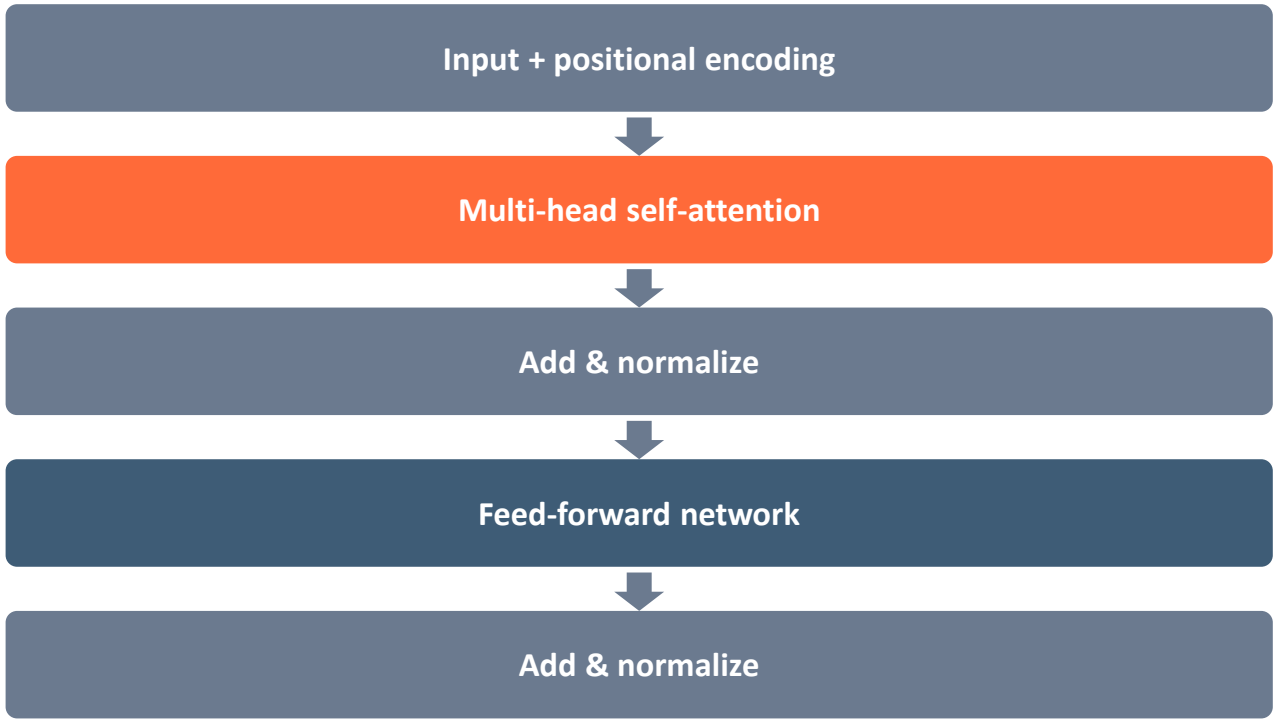
Usually it is not

Ask “how long ago was the impact?” or “did the overload come before or after the temperature rise?” and order becomes essential. Transformers add a positional encoding to every input so position is part of the representation.

A useful contrast to carry away: the LSTM gets order for free and long-range access with difficulty. Attention gets long-range access for free and order only if you add it.

The Transformer Block

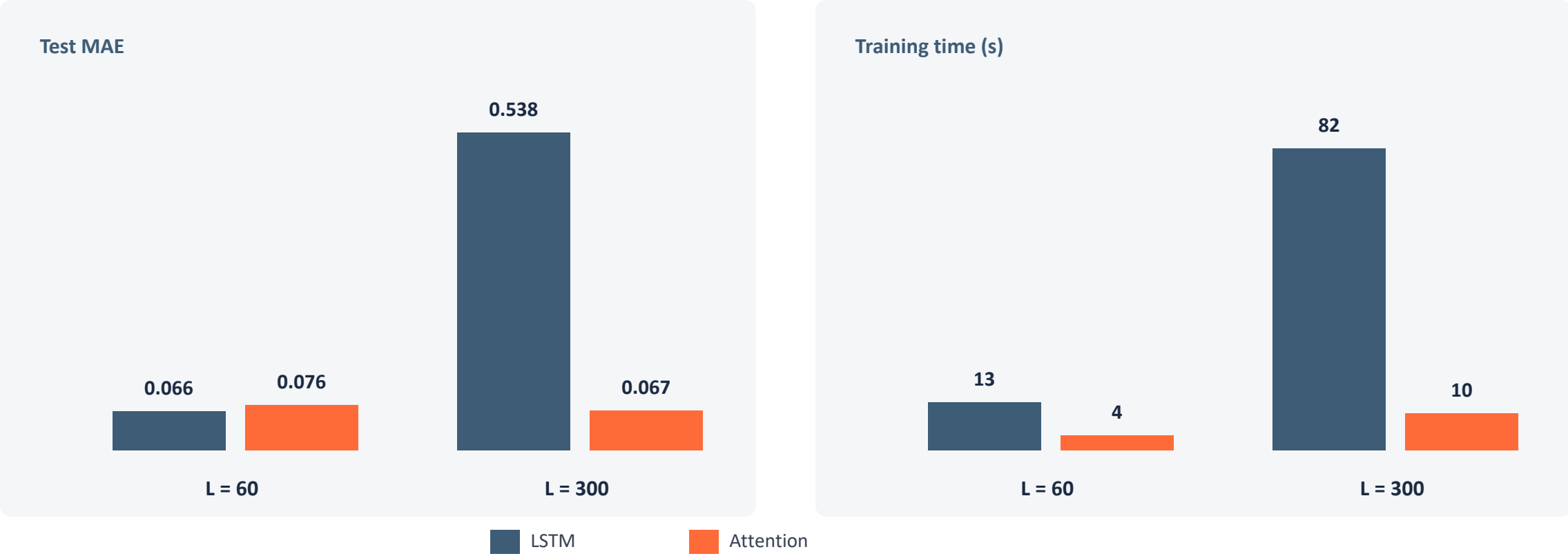
Stack self-attention with a small feed-forward network, wrap both in residual connections and normalization, and repeat. That is the whole architecture.



Self-attention means Q, K and V all come from the same sequence. Every position attends to every other, so information crosses the whole sequence in a single layer — no matter how long it is.

The Result: It Depends on Length

Both models, same data, same optimizer, same 40-epoch budget. Only the sequence length changes.



At L = 60 the LSTM is slightly ahead. Attention is not magic on short sequences — a hidden state copes fine over sixty steps. At L = 300 the same budget gives eight times the error and eight times the training time. Run at four lengths, the LSTM is fine at 60 and fails from about 120 onward.

Is the LSTM Just Under-Trained?

A fair objection, and worth testing rather than assuming. Give the LSTM two and a half times the training budget at L = 300 and see what happens.

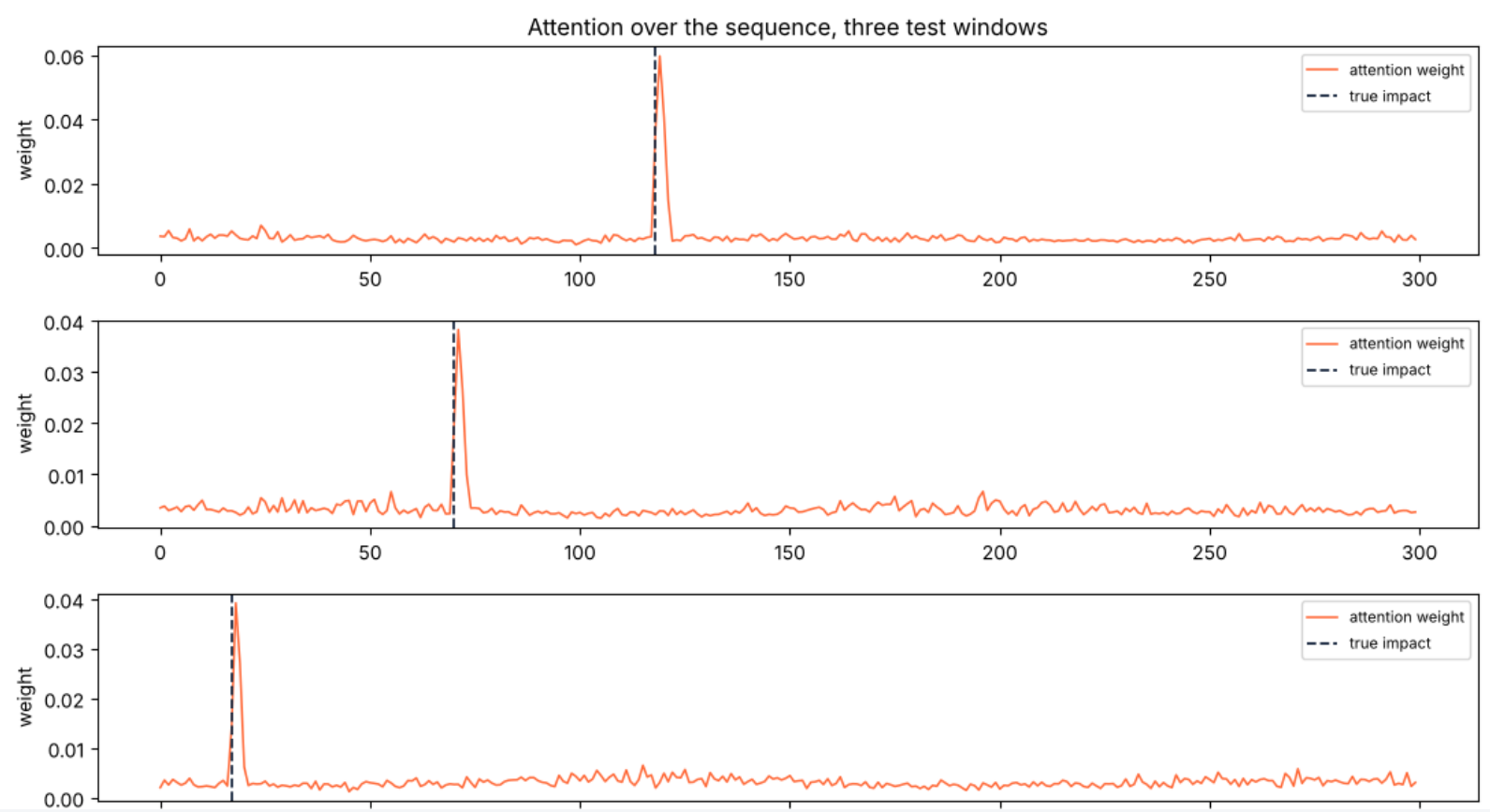
Attention, 40 epochs	MAE 0.067	10 s
LSTM, 40 epochs	MAE 0.538	82 s
LSTM, 100 epochs	MAE 0.223	190 s

More training does help. The LSTM is not incapable — its error falls by more than half.

But it is still around three times worse, having spent seventeen times attention's training time.

That is the honest claim: not “cannot”, but “costs far more to get less”.

Looking Inside: What Did It Attend To?



One query means one weight per time step — so the model's reasoning is a single curve you can plot.

The peak lands within three steps of the real impact in 100% of test windows.

The impact position was never a training label. The model had to find it to solve the task.

Week 9 showed you what patterns a CNN had learned. This shows you where in the input the model chose to look — and you can check it against ground truth you never trained on. For a safety case, a model that can point at the event responsible is far easier to trust.

Where This Lands in Robotics



Vision Transformers

Cut an image into 16×16 patches, treat each patch as a token, and run self-attention over them. Same mechanism you just built — the sequence is spatial instead of temporal.



Sensor fusion

Camera, lidar, force and joint telemetry become one sequence of tokens. Attention learns which modality to trust for a given decision, instead of a fixed hand-tuned weighting.



The cost: $O(L^2)$

Every position attends to every other, so doubling the sequence quadruples the work. This is the active research frontier and the reason very long logs are still hard.

Worth keeping in proportion: transformers dominate where data is plentiful and sequences are long. On a few thousand rows of tabular sensor data, Week 4's gradient boosting is still the right answer.

Today's Lab: Build Attention From Scratch

1	**Implement the equation yourself**	Then verify it matches PyTorch's own, to five decimals.
2	**Wire up queries, keys and values**	A single learned query pooling over the sequence.
3	**Run the length experiment**	L = 60 and L = 300, same budget, both models.
4	**Plot what the model looked at**	And check the peak against the true impact position.

Key Takeaways

- Attention replaces a passed-along state with direct access to every position.
- On short sequences it buys you nothing. The advantage grows with length.
- It is order-blind unless you add position — the mirror image of an RNN.

NEXT WEEK

Reinforcement Learning

No labelled answers at all — a policy that learns which action to take from reward alone.